MODULE 06 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

# Lambda IV – Streaming, Edge and What Changed

Function URLs, response streaming, edge compute – and four rules the exam still teaches that AWS has quietly changed.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 6 of 34

admin@sthithapragna.com

A Lambda function does nothing until something calls it. This module is about the front door - and there are now four of them.

# Lambda function URLs

- A dedicated HTTPS endpoint on the function: **https://<id>.lambda-url.<region>.on.aws**
- **No additional charge.** You pay only for the Lambda invocation.
- Built-in CORS configuration, no code required.
- Can be attached to a specific **alias or version**, using the qualifier.
- One function, one URL. It is not an API, and it has no routing.

> ON THE EXAM
>
> Function URLs are for a single-purpose endpoint. The moment you need paths, methods and routing, you want API Gateway.

# The function URL auth model

- **AWS_IAM** - the caller must sign the request with SigV4 and hold lambda:InvokeFunctionUrl.
- **NONE** - completely public. Anyone with the URL can invoke it.
- No API keys, no usage plans, no per-caller throttling, no Cognito authoriser.
- No Lambda authorisers either. Custom auth must live inside your handler.
- For WAF or per-caller rate limiting, put CloudFront or API Gateway in front.

> ON THE EXAM
>
> The absence of API keys and usage plans is the fact that decides questions. Any scenario mentioning tiered access or per-customer quotas rules function URLs out immediately.

# Function URL or API Gateway?

FUNCTION URL

## One endpoint, no management

- No extra charge at all
- IAM or public. Nothing else
- No routing or path matching
- Supports response streaming
- Set up in one configuration step

API GATEWAY

## A managed API surface

- Charged per request
- API keys, usage plans, authorisers
- Routing, stages, versioning
- Request validation and caching
- WAF attaches directly

THE TIEBREAKER

If the scenario names throttling per customer, API keys, request validation or several routes, it is API Gateway. If it says least operational overhead for one internal endpoint, it is a function URL.

# CloudFront in front of a function URL

- CloudFront fixes most of what a function URL lacks: caching, **WAF**, Shield, a custom domain.
- **Origin access control** for function URLs, since April 2024, signs origin requests with SigV4.
- That lets the function URL use **AWS_IAM** and stay unreachable directly.
- Without OAC, the URL had to be public and the function had to check a shared secret itself.
- Streaming still works through CloudFront, so you keep both.

> ON THE EXAM
>
> A function URL with AuthType NONE is genuinely open to the internet. CloudFront with OAC is how you front one safely without moving to API Gateway.

# Response streaming

- The function sends the response progressively instead of buffering it whole.
- **Time to first byte** drops sharply. The user sees output while work continues.
- Node.js requires wrapping the handler in **streamifyResponse()**.
- Set **InvokeMode: RESPONSE_STREAM** on the function URL, or call InvokeWithResponseStream.
- Not supported on function URLs when the function is attached to a VPC.

> ON THE EXAM
>
> Two use cases dominate: generative AI token-by-token output, and payloads that exceed the 6 MB buffered limit.

# The streaming numbers

- **Buffered response: 6 MB.** The classic Lambda synchronous limit.
- **Streamed response: 20 MB** by default, and this one can be raised on request.
- The first **6 MB** streams at uncapped bandwidth.
- Beyond 6 MB, the remainder is capped at roughly **2 MB per second**.
- A disconnecting client does not stop the function. You pay for the full duration.

> ON THE EXAM
>
> Streaming raises the payload ceiling as well as improving latency. That second benefit is easy to forget and often the one a scenario is testing.

# Which front door actually streams

| FRONT DOOR | STREAMING SUPPORT |
| --- | --- |
| Lambda function URL | Yes, with InvokeMode RESPONSE_STREAM. Not inside a VPC |
| API Gateway REST API | Yes, since November 2025. See the next slide |
| API Gateway HTTP API | No. Buffered only |
| Application Load Balancer | No streaming path for Lambda targets |
| CloudFront with a function URL origin | Yes, and it adds caching and WAF |

ON THE EXAM

The HTTP API row surprises people, because HTTP APIs are usually the more modern option. On streaming, REST is the capable one.

# Currency note: REST APIs can now stream

- **The examinable answer:** API Gateway has a 29-second integration timeout and a 10 MB payload cap.
- Since **November 2025**, REST APIs support response streaming.
- That extends integration timeouts to **15 minutes** and allows payloads **over 10 MB**.
- Works with Lambda, HTTP proxy and private integrations. REST only, not HTTP APIs.
- **Answer 29 seconds and 10 MB on the exam.** Exam guide v2.1 predates this by a year.

> ON THE EXAM
>
> Two of the most reliable API Gateway facts became conditional. Before November 2025 the only way to stream from a REST surface was a function URL behind an HTTP_PROXY integration.

# Lambda@Edge

- Node.js or Python, running at CloudFront **regional edge caches**.
- All **four** trigger points, including origin request and origin response.
- Network access, filesystem access, and it can call other AWS services.
- Must be created in **us-east-1** and deployed from there to the edge.
- No environment variables, no VPC access, no layers. Deployment takes minutes.

> ON THE EXAM
>
> The three missing features catch people out. Configuration cannot come from environment variables, so it is hardcoded or fetched at runtime and cached.

# CloudFront Functions

- JavaScript only, running at **CloudFront edge locations** themselves.
- **Viewer request and viewer response only.** No origin triggers.
- Sub-millisecond execution, and roughly a sixth the price of Lambda@Edge.
- **2 MB memory, 10 KB of code, under 1 ms.** Severe by design.
- No network access, no filesystem, no access to the request body.

> ON THE EXAM
>
> The 'no network access' limit is the cleanest eliminator in the pair. If the code must look anything up, it cannot be a CloudFront Function.

# CloudFront Functions or Lambda@Edge?

CLOUDFRONT FUNCTIONS

## Rewrite, redirect, header work

- URL rewrites and redirects
- Header manipulation
- Cache key normalisation
- Simple token checks
- Millions of requests, lowest cost

LAMBDA@EDGE

## Anything that needs to think

- Calls another service or API
- Reads or changes the request body
- Origin request or origin response
- Image resizing, content generation
- Needs more than 10 KB of code

THE TIEBREAKER

Three questions decide it: does it need the network, does it need an origin trigger, and is the code over 10 KB? Any yes means Lambda@Edge.

# The four CloudFront trigger points

| TRIGGER | WHEN IT RUNS, AND WHAT IT IS FOR |
| --- | --- |
| Viewer request | Every request, before the cache is checked. Auth, rewrites, cache keys |
| Origin request | Only on a cache miss, before going to the origin. Origin selection, lookups |
| Origin response | Only on a cache miss, after the origin replies. Transforming content |
| Viewer response | Every response, before it reaches the user. Security headers |

ON THE EXAM

Viewer triggers run on every request. Origin triggers run only on cache misses, so they are cheaper and can safely do more work.

# Lambda behind an Application Load Balancer

- A Lambda function can be a **target group** on an ALB, alongside EC2 and containers.
- Useful when routing rules already live in an ALB, or during a gradual migration.
- **1 MB payload limit** in both directions – far smaller than API Gateway's 10 MB.
- No response streaming, no API keys, no usage plans, no request validation.
- The ALB is billed hourly, so it only makes sense if one already exists.

> ON THE EXAM
>
> The realistic use case is a migration: existing traffic already flows through an ALB, and one path is being moved to Lambda without redesigning the routing.

# Currency note: Lambda Durable Functions

- **The examinable rule:** Lambda stops at 15 minutes, so longer workflows need Step Functions.
- Durable Functions let a function run or wait for **up to one year**.
- You write ordinary sequential code with a durable context object and checkpoints.
- **context.wait()** suspends the execution, and waiting time is not charged.
- **Not in exam guide v2.1.** Answer 15 minutes and Step Functions on the day.

> READ THIS TWICE
>
> This is the single most useful thing to know about modern Lambda and the single most dangerous thing to answer with. The exam's rule is unchanged.

# Durable Functions or Step Functions?

STEP FUNCTIONS

## The examinable answer

- Workflow defined outside your code
- Visual state machine and history
- Service integrations without code
- Priced per state transition
- What the exam expects

DURABLE FUNCTIONS

## The 2026 reality

- Workflow written as ordinary code
- Automatic checkpoint and replay
- Up to one year, idle time unbilled
- Node.js and Python at launch
- Not examinable on DVA-C02

THE TIEBREAKER

Included so you are not confused when you meet it at work. On the exam, a long-running or multi-step workflow is Step Functions. Module 13 covers that properly.

# The revision gap, in one table

| THE EXAM STILL SAYS | AWS NOW ALSO OFFERS |
| --- | --- |
| Lambda hard-stops at 15 minutes | Durable Functions: up to one year, idle unbilled |
| API Gateway integration timeout is 29 seconds | REST response streaming: up to 15 minutes |
| API Gateway payloads cap at 10 MB | REST response streaming: larger payloads |
| One execution environment per concurrent request | Managed Instances: shared, EC2-backed |

ON THE EXAM

Answer the left column on exam day. Expect the right column when you open current AWS documentation. Module 31 collects every one of these.

Two of these turn on a constraint you have to **notice** rather than calculate. Read the second paragraph carefully.

A company runs a customer support assistant on Amazon Bedrock behind a Lambda function. Responses take 20 to 40 seconds to generate fully, and users currently stare at a spinner until the whole answer arrives. The team wants tokens to appear as they are produced.

The assistant is one route on an existing API Gateway REST API. Access is authorised by a Cognito user pool authoriser, and each customer tier has a different request quota enforced by a usage plan. Both must be preserved.

## How should the team deliver the streamed response?

A. Replace the route with a Lambda function URL using InvokeMode RESPONSE_STREAM

B. Enable response streaming on the existing API Gateway REST API route

C. Migrate the API to an API Gateway HTTP API and enable response streaming

D. Put the response in Amazon S3 and return a presigned URL for the client to poll

◆ PAUSE AND DECIDE

# How should the team deliver the streamed response?

**A** Function URL with RESPONSE_STREAM

*Streams correctly, but function URLs support only IAM or public auth and have no usage plans. Both stated requirements would be lost.*

**B** Response streaming on the REST API

Keeps the Cognito authoriser and the usage plans, keeps the existing route, and REST APIs have supported response streaming since November 2025.

**C** Migrate to an HTTP API

*HTTP APIs do not support response streaming at all, and migrating would also lose usage plans, which HTTP APIs lack.*

**D** S3 plus polling

*A pre-streaming workaround. It does not improve time to first byte at all - the user still waits for the whole answer, now with polling on top.*

THE SIGNAL

Three constraints: stream, keep Cognito, keep usage plans. Only one option holds all three - and only because of a change made in November 2025.

A media site serves images through Amazon CloudFront. The team wants to normalise the cache key by lowercasing the path and stripping tracking query parameters, and to add a set of security headers to every response.

The distribution serves about 2 billion requests a month. The logic is roughly 40 lines of JavaScript, needs no external data, and must add as little latency as possible.

## How should this be implemented?

A. A Lambda@Edge function on the origin request and origin response triggers

B. A Lambda@Edge function on the viewer request and viewer response triggers

C. CloudFront Functions on the viewer request and viewer response triggers

D. A CloudFront response headers policy plus a Lambda@Edge viewer request function

◆ PAUSE AND DECIDE

## How should this be implemented?

**A** Lambda@Edge on origin triggers

*Origin triggers only fire on cache misses. Cache key normalisation must happen before the cache is checked, so it would never run for cached requests.*

**B** Lambda@Edge on viewer triggers

*It would work, at roughly six times the cost across 2 billion requests, with higher latency and no capability actually being used.*

**C** CloudFront Functions on viewer triggers

No network access needed, code well under 10 KB, viewer triggers only - it fits every constraint, at the lowest cost and latency.

**D** Headers policy plus Lambda@Edge

*The headers policy half is genuinely good practice. Pairing it with Lambda@Edge for a simple rewrite still overpays for the rewrite.*

THE SIGNAL

Option A is the real trap. Cache key normalisation has to happen at viewer request, because by origin request the cache has already been consulted.

> An internal reporting tool needs to trigger a Lambda function over HTTPS. The only callers are three other services inside the same AWS account, all of which already use the AWS SDK and have IAM roles.
>
> There are no plans for external access, no need for routing beyond a single endpoint, and no per-caller quotas. The platform team has asked for the option with the least operational overhead and the lowest cost.

## What should the developer implement?

A. An API Gateway REST API with IAM authorisation

B. An API Gateway HTTP API with a JWT authoriser

C. An Application Load Balancer with the function as a target group

D. A Lambda function URL with AuthType AWS_IAM

◆ PAUSE AND DECIDE

## What should the developer implement?

**A** REST API with IAM auth

*Correct and secure, but you pay per request for routing, stages and usage plans that the scenario explicitly says are not needed.*

**B** HTTP API with JWT authoriser

*Introduces a token issuer the callers do not have. They already hold IAM roles, so JWT adds a moving part for nothing.*

**C** ALB target group

*An ALB bills hourly whether used or not, and none is mentioned as existing. The most expensive option here by some distance.*

**D** Function URL with AWS_IAM

One endpoint, SigV4 auth the callers can already produce, no extra charge, and a single configuration step.

THE SIGNAL

This scenario is the mirror image of scenario 1. Same choice, opposite answer, because every managed API feature was explicitly ruled out.

A retailer serves product pages through CloudFront. When a product is discontinued, requests for its URL must be redirected to the replacement product. The mapping of old URL to new URL lives in a DynamoDB table and changes several times a day.

Redirects apply to roughly 3% of traffic. The team wants the lookup to happen only when CloudFront does not already have the page cached.

## Where should the redirect logic run?

A. A Lambda@Edge function on the origin request trigger

B. A CloudFront Function on the viewer request trigger

C. A Lambda@Edge function on the viewer request trigger

D. A CloudFront Function on the viewer request trigger, with the mapping in the function code

◆ PAUSE AND DECIDE

# Where should the redirect logic run?

**A** Lambda@Edge on origin request

Only Lambda@Edge can query DynamoDB, and the origin request trigger fires only on cache misses - exactly the behaviour asked for.

**B** CloudFront Function on viewer request

*CloudFront Functions have no network access, so the DynamoDB lookup is impossible. This is the cleanest disqualification in the pair.*

**C** Lambda@Edge on viewer request

*Capable of the lookup, but viewer request fires on every request including cache hits. It contradicts the stated requirement and costs far more.*

**D** CloudFront Function with the mapping in code

*Sidesteps the network limit, but the 10 KB code limit will not hold a product mapping, and it must be redeployed several times a day.*

THE SIGNAL

Two constraints, two eliminations. 'Lives in DynamoDB' removes CloudFront Functions. 'Only when not cached' removes viewer triggers.

# What to carry forward

- Function URLs offer **IAM or public**. No API keys, no usage plans, no authorisers.
- Streaming: 6 MB buffered, 20 MB streamed. REST APIs stream, HTTP APIs do not.
- Edge: **network access, origin trigger, or over 10 KB** means Lambda@Edge.
- Viewer triggers run on every request. Origin triggers run only on cache misses.
- Four exam rules now have exceptions in the live service. **Answer the exam rule.**

> ON THE EXAM
>
> That completes Lambda. Next: DynamoDB, starting with keys, partitions and indexes - the second most tested service on this exam.